

客户端之HTML

什么是HTML?

HTML 即 Hyper Text Markup Language, 指超文本标记语言而不是编程语言, 是使用标记标签来描述网页的一种语言

(HTML5 是 HTML 的第5次重大修改, 希望减少互联网应用(RIA)对第三方插件的依赖, 并且提供更多能有效增强网络应用的API.)

HTML5 的8大新特性: 语义, 离线存储, 新表单功能, 连接, 多媒体, 三维图形与特效, 性能与集成, CSS3)

HTML 文件结构:

`<!DOCTYPE html>` ← 声明浏览器文档使用的HTML规范, 按此标准渲染页面

`<html lang="en">` ← HTML 开始标签

`<head>` ← 首部标签

`<meta charset="UTF-8">`

`<meta name="viewport" content="width=device-width, initial-scale=1.0">` → meta 标记可以指定 HTML5 的字符集

`<title>` 页面标题 `</title>`

网页标题标签

`<style>` ← 样式定义标签

内部CSS样式

`</style>`

`</head>`

`<!--HTML注释-->`

`<body>` ← 主体标签

`<div>` ← 通用区块容器标签

分组

`</div>`

`<script>` ← 脚本标签

JavaScript 代码

`</script>`

`</body>`

`</html>` ← HTML 结束标签

HTML 文档有3种编写方法:

① 手工在记事本直接编写, 存成 .html 格式 ② HTML 编辑器 ③ Web(HTTP)服务器实时动态生成

(1) HTML 标题

通过 `<h1>` ~ `<h6>` 标签来定义, 数字越小, 标题级别越高, 字体越大

它们都是**块级元素**, 默认独占一行, 并且文字粗体显示
并非单纯用于调节字体大小, 而具有表示内容层级关系的语义

`<title>` 和 `<h1>` 的区别?

- ① `<title>` 位于 `<head>` 中, 显示在浏览器的标签栏, 并且1个html文档只能有1个 `<title>` 标签, 内容是纯文本, **不能嵌套其它标签**
- ② `<h1>` 位于 `<body>` 中, 显示在网页的可见内容区域

(2) HTML 段落

`<span>` : 行内元素

通过 `<p>` 标签来定义 `<p>` 这是普通文字 `<span style="color:red">红色词</span>` 其它它是**块级元素**, 默认独占一行 都普通。`</p>`

多个 `<p>` 相邻时, 会存在默认的上下外边距 (margin), 段落之间存在间隙

`<p>` 内可以包含文本以及行内元素 (如 `<a>` `<span>` `<em>`), 但**不能嵌套块级元素**

`<p>` 和普通文本的区别?

- ① `<p>` 包裹的文本是块级段落, 自动独占一行, 段落之间有默认间距
- ② 直接写的普通文本是行内文本, 不会自动换行, 段落之间无默认间距

`<p>` 和 `<br>` 的区别?
换行

- ① `<p>` 是块级双标签, 有表示段落的语义, 有默认的上下外边距, 段间有间隙
- ② `<br>` 是行内单标签, 仅实现同一文本块内的强制换行, 没有额外的间隙

`<p>` 的默认 CSS 样式是: `p {`

`display: block;` // 块级显示模式
`width: 100%;` // 宽度默认撑满父容器, 可手动设置宽/高
`margin: 1em 0;` // 让段落之间产生等于字体大小的上下间距
改成 "inline" 就变成行内模式, 不可手动设置宽/高, 全由内容撑开
改成 "inline-block" 则既可并排显示又可手动设置宽/高

相对当前元素的字体大小 (font-size)
浏览器默认的 font-size 是 16px
若当前 font-size = 20px, 则 1em = 20px

文字格式控制: `h5` 强调标签语义化

- `<i>` 倾斜文本 `</i>` → `<em>` 倾斜文本 `</em>`
- `<b>` 粗体文本 `</b>` → `<strong>` 粗体文本 `</strong>`
- `<u>` 下划线文本 `</u>` → `<ins>` 下划线文本 `</ins>`
- `<s>` 删除线文本 `</s>` → `<del>` 删除线文本 `</del>`

(3) HTML 链接

通过<a> 标签来定义

例如 `<a href="http://www.ahut.edu.cn">` 这是一个链接
它是行内元素, 默认样式为带下划线的蓝色文字, 点击后变为紫色文字
指定链接的目标地址 "...html" 亦可
设置 `a { text-decoration: none; }` 可以去掉超链接默认的下划线
默认 `target="self"` (在当前标签页打开) `target="_blank"` (在新标签页打开)

(4) HTML 图片

通过 标签来定义

例如 ``
若只设置1个, 则图片会自动等比例缩放另一个
`alt="这是一张图片"`

它是行内替换元素, 图片的名称和尺寸以属性的形式提供 (并不独占一行)

可以设置 alt 属性, 若 src 路径正确则显示图片, 反之会在图片位置显示替代文本

 与普通行内元素的区别?

① 可以手动设置宽/高, 水平/垂直方向的内/外边距, 内容由外部资源替换, 与父级容器的基线对齐 (常致底部缝隙)
设置: `img { vertical-align: middle; }`
`margin-right, margin-left, padding-right, padding-left, margin-top, margin-bottom, padding-top, padding-bottom`

② 普通行内元素不可手动设置宽/高, 由自身内容撑开, 只能设置水平方向的内/外边距而不能设置垂直方向的, 内容就是标签内的文本内容, 与父级文本的基线对齐

(默认 baseline, top 是与行内最高元素顶部对齐, bottom 是与行内最低元素底部对齐, middle 是元素中间与父元素的基线 + "x" 高度的一半"对齐)

(5) HTML 音频

HTML5 中的新属性

行内元素

(6) HTML 视频

HTML5 中的新属性 (同(5)者省略)

属性	值	描述
autoplay	autoplay	就绪后立即播放
controls	controls	向用户显示控件 (如播放按钮)
loop	loop	结束时重播
muted	muted	静音播放
preload	preload	页面加载时同步加载, 预备播放
src	url	要被播放者的 url

属性	值	属性
height	pixels	设置播放器高度
poster	url	规定视频下载时或用户点击播放按钮前显示的图像
width	pixels	设置播放器宽度

`<audio src="music.mp3" controls style="width:100%;">`

`<video src="video.mp4" controls style="display:block; margin: 0 auto;">`

注: 若同时设置 autoplay 和 preload, preload 被忽略

路径 { 绝对路径: 完整的路径
相对路径: 相对点 (./ 表示当前目录, ../ 表示上一级目录)

列表

无序列表 *ul, li, ol, dl, dt, dd, 全是块级元素*

去掉小黑点? list-style-type: none;

无序列表是一个项目的列表, 该项目使用**粗体圆点**进行标记

无序列表始于标签, 每个列表项始于标签

例如

<pre> Coffee Milk </pre>		• Coffee • Milk
---	--	---

有序列表

列表项目使用**数字**进行标记

有序列表始于标签, 每个列表项始于标签

例如

<pre> Coffee Milk </pre>		1. Coffee 2. Milk
---	--	---

自定义列表

自定义列表是项目及其注释的组合

自定义列表始于<dl>标签, 每个自定义列表项始于<dt>, 每个自定义列表项的定义始于<dd>

例.

本学期所学课程如下:

1. 离散数学
2. 大学物理
3. 数字逻辑

在页面水平居中:

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<meta charset="UTF-8">
```

```
<title>有序列表的建立</title>
```

```
<style>
```

```
.course-list {  
    border: 1px solid red;  
    padding: 15px;  
    width: 300px;  
    margin: 0 auto;★  
}
```

```
ol {  
    padding-left: 40px;  
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<div class="course-list">
```

本学期所学课程如下:

```
<ol>
```

```
<li>离散数学</li>
```

```
<li>大学物理</li>
```

```
<li>数字逻辑</li>
```

```
</ol>
```

```
</div>
```

```
</body>
```

```
</html>
```

在页面
垂直居中:

```
body {  
    margin: 0; //清除body默认边距  
    height: 100vh; //body高度=浏览器视口高度  
    display: flex; //开启flex布局  
    justify-content: center; //子元素水平居中  
    align-items: center; //子元素垂直居中  
}  
.course-list {  
    border: 1px solid red;  
    padding: 15px;  
    width: 300px;  
}
```

表格

一个表格可以分成很多行, 用<tr>表示

每行可以分成很多单元格 (cell), 用<td>表示

例如

`<table border = "1">`
`<tr>` ← 行
 `<td> row1, cell1 </td>` ← 单元格
 `<td> row1, cell2 </td>`
`</tr>`
`<tr>`
 `<td> row2, cell1 </td>`
 `<td> row2, cell2 </td>`
`</tr>`
`</table>`

row1, cell1	row1, cell2
row2, cell1	row2, cell2

要设置表格的样式,

方法一: 写在<table>标签内 (内联)

例如 `<table style = "border: 3px solid green; border-collapse: collapse; ">`
边框 宽 实线 颜色 合并单元格边框

方法二: 写在<style>标签内 (css)

例如 `<style>`

`table {`

`border: 3px solid green; // 表格外边框`

`border-collapse: collapse; // 合并单元格边框`

`}`

`td {`

`border: 1px solid green; // 单元格边框`

原 cellpadding — `padding: 10px; // 单元格内边距`

`</style>`

注: 如果希望设置单元格之间的间距, "border-collapse: separate;" (默认),

再添加 "border-spacing: 2px;" 即可

原 cellspacing

10px 16px
水平间距 垂直间距

例.

成绩表 color: #ce2c2c

姓名	数学	语文
小王	99	100
小明	100	100
制表日期: 2025-12-5		

<!DOCTYPE html>

<html>

<head>

<title> 例题练习 </title>

<style>

caption {

font-size: 24px;

font-weight: bold;

color: #ce2c2c;

margin-top: 20px; // 标题顶部到表格容器顶部

margin-bottom: 10px; // 标题底部到表格内容顶部

}

table {

border-collapse: collapse;

margin: 20px auto;

width: 600px;

}

th, td {

border: 1px solid #333;

padding: 10px 20px;

text-align: center;

}

.right-align {

text-align: right;

}

</style> (如文字, span, img) 水平方向的对齐, 给行内元素

</head> 设置无效而只有给行内元素的块级父容器设置才有效

顺序: "上→右→下→左" 或 "上下 左右"

margin-top/right/bottom/left:

当前元素上/右/下/左边缘与该方向最近相邻元素的下/左/上/右边缘的距离

限制: 行内元素的上/下外边距不生效

(替换元素, 例如img/input, 4个方向都生效)

padding-top/right/bottom/left:

当前元素上/右/下/左边缘与元素自身内容的上/右/下/左边缘的距离

限制: 行内元素的上/下内边距不影响外部布局, 仅"撑"自身

(替换元素4个方向的内边距都影响布局)

<body>

<table> ← 表格标签

表格标题标签, → <caption> 成绩表 </caption> 嵌套在<table>内部, 与表格居中对齐

<tr> ← 表格行标签

表头单元格标签, <th> 姓名 </th>

文字居中且加粗 <th> 数学 </th>

<th> 语文 </th>

</tr>

<tr>

单元格标签 (容器级元素, 可放任何东西)

<td> 小王 </td>

<td> 99 </td>

<td> 100 </td>

</tr>

<tr>

<td> 小明 </td>

<td> 100 </td>

<td> 100 </td>

</tr>

<tr>

<td colspan="3">

class="right-align">

制表日期: 2025-12-5

</td>

</tr>

</table>

</body>

</html>

div, p, h1~h6, table, ul/li, form 都是块级元素 display: block;

span, a, em, strong, br 都是行内元素

display: inline;

表单

用表单标签 <form> 来定义, 目的是为收集用户信息, 可以与用户交互

<form> → 块级元素

控件标签 {
input 标签
select 下拉菜单
textarea 文本域
label 标签

> 行内替换元素, 不独占一行, 可设宽/高
→ 20个英文字符10个中文字符
— 块级元素, 默认有固定行(2)列(20), 独占一行, 可设宽/高
— 行内元素, 不独占一行, 不可设宽高(需转换为块级/行内块)

</form>

表单域: form

表单属性

① <input 属性="属性值"> (输入控件)

①~④ 都有的:

属性=type	属性值	描述
老	text	文本框
	password	密码框
	radio	单选框
	checkbox	复选框
	button	普通按钮
	submit	提交按钮
	reset	重置按钮
	image	图像按钮
	file	文件域
新	email	限制输入为邮箱格式
	url	限制输入为网址格式
	date	弹出日期选择器(YYYY-MM-DD)
	time	弹出时间选择器(HH:MM:SS)
	month	弹出月份选择器(YYYY-MM)
	week	弹出周选择器(YYYY-WW)
	number	仅允许输入数字
	tel	弹出数字键盘
	search	搜索框(带清除按钮)
	color	弹出颜色选择器

属性	属性值	描述
name	自定义	控件的名称
value	自定义	input控件中默认文本值
size	正整数	input控件在页面中显示宽度
checked	checked	定义选择控件默认选中的项
maxlength	正整数	控制允许输入最多字符数

② <select 属性="属性值"> (下拉表单)

```
<select>  
  <option>  
  </option>  
</select>
```

③ <textarea 属性="属性值"> (文本域)

```
<textarea>  
</textarea>
```

④ <label 属性="属性值"> (标记表单控件的标签)

```
<label>  
</label>
```

密码输入框点击“小眼睛”切换明文与密文

```
<!DOCTYPE html>
<html lang="zh-CN">
```

```
<head>
```

```
<meta charset="UTF-8">
```

```
<title>密码的显示与隐藏</title>
```

```
<style>
```

```
  .pwd-container {
```

```
    position: relative; // 父容器相对定位
```

```
    width: 300px;
```

```
    margin: 50px auto;
```

```
  #pwd-input {
```

```
    width: 100%; // 占满父容器宽度
```

```
    padding: 10px 40px 10px 10px;
```

```
    font-size: 16px; // 就是胶囊, 50%是椭圆
```

```
    border: 1px solid #ccc; // 1 2 3 4 2值就是 "1.3" "2.4" 共用
```

```
    border-radius: 4px; // 圆角 3值就是 "1" "2.4" "3" 共用
```

```
    box-sizing: border-box; // 盒子模型, 包含padding, border
```

```
  } // 实际就是300px else {
```

```
  .eye-icon {
```

```
    position: absolute; // 不然超出会变成352px
```

```
    right: 10px; // 子元素绝对定位(相对于父元素)
```

```
    top: 50%; // 顶部距父容器顶部距离: 父容器高度50%
```

```
    transform: translateY(-50%); // 元素向上移动其自身高度的50%
```

```
    cursor: pointer; // 鼠标样式: 手型
```

```
    font-size: 20px;
```

```
    color: #999;
```

```
    user-select: none;
```

```
  } // 伪类选择器
```

```
  .eye-icon: hover {
```

```
    color: red; // 鼠标悬浮时的颜色
```

```
    cursor: default; // 默认值, 箭头
```

```
  } // 属性名
```

```
  pointer // 小手 提示点击
```

```
  text // 工字形 提示选择文本
```

```
  move // 十字形 提示移动
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<div class="pwd-container">
```

```
<input type="password" id="pwd-input" placeholder="请输入">
```

```
<span class="eye-icon" id="eye-btn">👁️</span>
```

```
</div>
```

```
<script>
```

```
const pwdInput=document.getElementById('pwd-input');
```

```
const eyeBtn=document.getElementById('eye-btn');
```

```
let isShow=false;
```

```
eyeBtn.addEventListener('click',function(){
```

```
if(isShow){
```

```
pwdInput.type='password';
```

```
eyeBtn.textContent='👁️';
```

```
isShow=false;
```

```
else{
```

```
pwdInput.type='text';
```

```
eyeBtn.textContent='👁️';
```

```
isShow=true;
```

```
});
```

```
</script>
```

```
</body>
```

```
</html>
```

设置50%:

宽高相等时是正圆

宽高不等时是椭圆

设置999px:

“左右半圆”

上下直边”的

胶囊形态

相对定位: 相对于自己原来位置的偏移, 原来所占空间仍然保留, 周围元素不补位

绝对定位: 相对于最近的已定位祖先元素的绝对偏移

框架 <frameset>

通过使用框架,可以在同一个浏览器窗口中显示不止一个页面

html5 中用的是 iframe 标签而 frameset 逐渐被弃用,iframe 可以插入页面中任何地方

注意! 不能将 <body></body> 标签与 <frameset></frameset> 标签同时使用!

<!DOCTYPE html>

<html lang="zh-CN">

<head>

<meta charset="UTF-8">

<title>HTML5-iframe 分栏布局</title>

<style>

清除浏览器
默认样式,并
统一盒子模型
的布局规则,避免
浏览器兼容与
布局错位

```
* {  
  margin: 0; 消除默认外边距,内边距,更容易  
  padding: 0; 计算和布局  
  box-sizing: border-box;  
}
```

body {

height: 100vh;

display: flex;

flex-direction: column;

}

• top {

height: 20%;

border: none;

}

• bottom {

flex: 1; // 自动填充剩余空间

display: flex;

}

• left {

width: 40%;

border: none;

}

• right {

flex: 1; // 自动填充剩余空间

border: none;

}

</style>

</head>

<body>

<iframe class="top" src="top.html" title="顶部"></iframe>

<div class="bottom">

<iframe class="left" src="left.html" title="左侧"></iframe>

<iframe class="right" src="right.html" title="右侧"></iframe>

</div>

</body>

</html>

启用合理的盒子模型:

border-box: width = 内容宽度 + padding + border

这就可以保证设置 padding 和 border 时不影响元素的实际宽度

默认值 content-box 的宽高仅包含内容区

总宽高 = 宽高 + padding + border

总宽高 = 宽高

宽高包含了 padding 和 border

版心居中

将页面的核心内容区域固定宽度后,水平居中显示在浏览器窗口中

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
* {  
  margin: 0;  
  padding: 0;  
  box-sizing: border-box;  
}
```

清除浏览器默认自带的边距

```
.container {
```

```
  width: 1200px;
```

```
  margin: 0 auto;
```

```
  border: 1px solid #ccc;
```

```
  padding: 20px;
```

```
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<div class = "container">
```

```
<h1> 标题 </h1>
```

```
<form>
```

```
<label> 姓名: </label>
```

```
<input type = "text">
```

```
</form>
```

```
</div>
```

```
</body>
```

```
</html>
```

上下边距为0,左右自动(水平居中)

注意 "margin: 0 auto" 生效的前提:

- 元素必须是块级元素或设置 `display: block;`
- 必须设置 `width` 或 `max-width` (不能是 `100%`).

那如何让文字在容器内部居中?

对于单行文字:

• container {

width: 300px;

height: 200px;

border: 1px solid green;

text-align: center; // 水平居中

line-height: 200px; // 垂直居中 (高度=容器高度)

}

line-height 的用法: ① 控制单行文字垂直居中

line-height 与 height 相等

对于多行文字:

• container {

width: 300px;

height: 200px;

border: 1px solid green;

display: flex;

justify-content: center; // 水平居中

align-items: center; // 垂直居中

}

② 控制多行文字行间距

line-height: 1.8

(等价于 1.8em, 即字体大小 × 1.8, 上下平均分配)

例. <!DOCTYPE html>

<html>

<head>

<style>

* {margin: 0; padding: 0; box-sizing: border-box;}

• container {

width: 300px; border: 1px solid pink; padding: 10px;

margin: 0 auto; // 版心居中

}

• text-box {

width: 200px; border: 1px solid green;

margin: 20px auto; // 版心中的容器居中

display: flex; justify-content: center; align-items: center; // 容器中的文字居中

}

</style>

</head>

<body>

<div class="container">

<div class="text-box">

文字

</div>

</div>

</body>

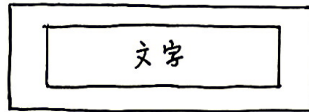
</html>

align-items: center; 是 Flex / Grid 容器的属性

控制子元素在容器内垂直居中

vertical-align: middle; 是元素自身的属性, 用于行内(块)元素

控制当前元素与同行元素垂直对齐



例.

```
<!DOCTYPE html>
<html lang="zh-CN">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
    content="width=device-width,
    initial-scale=1.0">
  <title>表单控件</title>
  <style>
    body {
      font-family: SimSun, serif;
      max-width: 800px;
      margin: 0 auto;
      padding: 20px;
      line-height: 1.6;
    }
    .form-group {
      margin-bottom: 20px;
      padding: 10px;
      border: 1px solid #ddd;
      border-radius: 5px;
    }
    .form-group label {
      display: inline-block;
      width: 120px;
      font-weight: bold;
    }
    .radio-group, .checkbox-group {
      display: inline;
    }
    .checkbox-group label {
      display: inline-block;
      margin-right: 15px;
    }
    input[type="submit"], input[type="reset"] {
      margin: 5px;
      padding: 8px 15px;
    }
    textarea {
      vertical-align: top;
    }
    .attribute-info {
      padding: 10px; margin: 10px 0; font-size: 0.9em;
    }
  </style>
</head>
```

form 的提交属性:

action=" " { ...: 表单提交的目标URL(后端接口)
: 临时占位, 用于本地测试
空: 默认提交至当前页面

method=" " { get: 数据拼在URL后
post: 数据在请求体中

文本输入框

密码

单选

复选

下拉菜单

文本

```
<body>
  <form action="#" method="post" enctype="multipart/form-data">
    <div class="form-group">
      <label for="username">用户名:</label>
      <input type="text" id="username" name="username" value="默认值" size="20"
        maxlength="10" placeholder="提示文本">
    </div>
    <div class="attribute-info"> 属性信息</div>
    <div class="form-group">
      <label for="password">密码:</label>
      <input type="password" id="password" name="password">
    </div>
    <div class="form-group">
      <label>性别:</label>
      <div class="radio-group">
        <input type="radio" name="gender" value="male" checked> 男
        <input type="radio" name="gender" value="female"> 女
      </div>
    </div>
    <div class="form-group">
      <label>爱好:</label>
      <div class="checkbox-group">
        <label><input type="checkbox" name="hobby" value="reading" checked> 阅读</label>
        <label><input type="checkbox" name="hobby" value="music"> 音乐</label>
        <label><input type="checkbox" name="hobby" value="travel"> 旅行</label>
      </div>
    </div>
    <div class="form-group">
      <label for="city">城市</label>
      <select id="city" name="city">
        <option value="shanghai" selected>上海</option>
        <option value="guangzhou">广州</option>
        <option value="shenzhen">深圳</option>
      </select>
    </div>
    <div class="form-group">
      <label for="comment">留言</label>
      <textarea id="comment" name="comment" rows="4" cols="100">
        默认留言内容
      </textarea>
    </div>
  </form>
```

表单数据的编码格式 (传文件必须用这个)

其值需与目标控件的id一致, 将label与目标表单控件相关联: 点label中的文字就能聚焦输入框

控件未输入时显示一段浅灰色提示文字

属性信息

强制隐藏: overflow: hidden;

强制显示垂直滚动条: textareas { overflow-y: scroll; }

默认留言内容 </textarea> wrap="off" 关闭自动换行
自适应, 内容超出"rows" "cols" 自动显示滚动条

target=" " { -self: 当前页面刷新
-blank: 新标签页打开
-parent/_top: 嵌套框架中用

普通按钮 {
<div class="form-group">
 <label> 普通按钮:</label>
 <input type="button" value="点击我" onclick="alert('这是按钮')">
</div>

提交按钮 {
<div class="form-group">
 <label> 提交按钮</label>
 <input type="submit" value="提交表单">
</div>

重置按钮 {
<div class="form-group">
 <label> 重置按钮</label>
 <input type="reset" value="重置表单">
</div>

日期 {
<div class="form-group">
 <label for="birthday">生日:</label>
 <input type="date" id="birthday" name="birthday" min="1899-12-01" max="2025-12-01">
</div>

电话 {
<div class="form-group">
 <label for="phone">电话:</label>
 <input type="tel" id="phone" name="phone" pattern="[0-9]{11}">
</div>

数字 {
<div class="form-group">
 <label for="age">年龄:</label>
 <input type="number" id="age" name="age" min="1" max="100" step="1" value="25">
</div>

搜索框 {
<div class="form-group">
 <label for="search">搜索:</label>
 <input type="search" id="search" name="search">
</div>

URL {
<div class="form-group">
 <label for="website">网站:</label>
 <input type="url" id="website" name="website">
</div>

邮箱 {
<div class="form-group">
 <label for="email">邮箱:</label>
 <input type="email" id="email" name="email" required>
</div>

文件上传 {
<div class="form-group">
 <label for="wenjian">上传文件:</label>
 <input type="file" id="wenjian" name="wenjian" accept="image/*">
</div>
</form>
</body>
</html>

*
image/jpeg, image/png
.pdf, .doc, .docx, .png

网页常见的布局方式：

① 标准流

块级元素，独占一行 → 垂直布局

行内(块)元素，一行显示多个 → 水平布局

② 浮动或 flex 等

让原本垂直布局的块级元素变成水平布局

③ 定位

让元素自由摆放在网页任意位置，一般用于盒子间的“层叠”情况